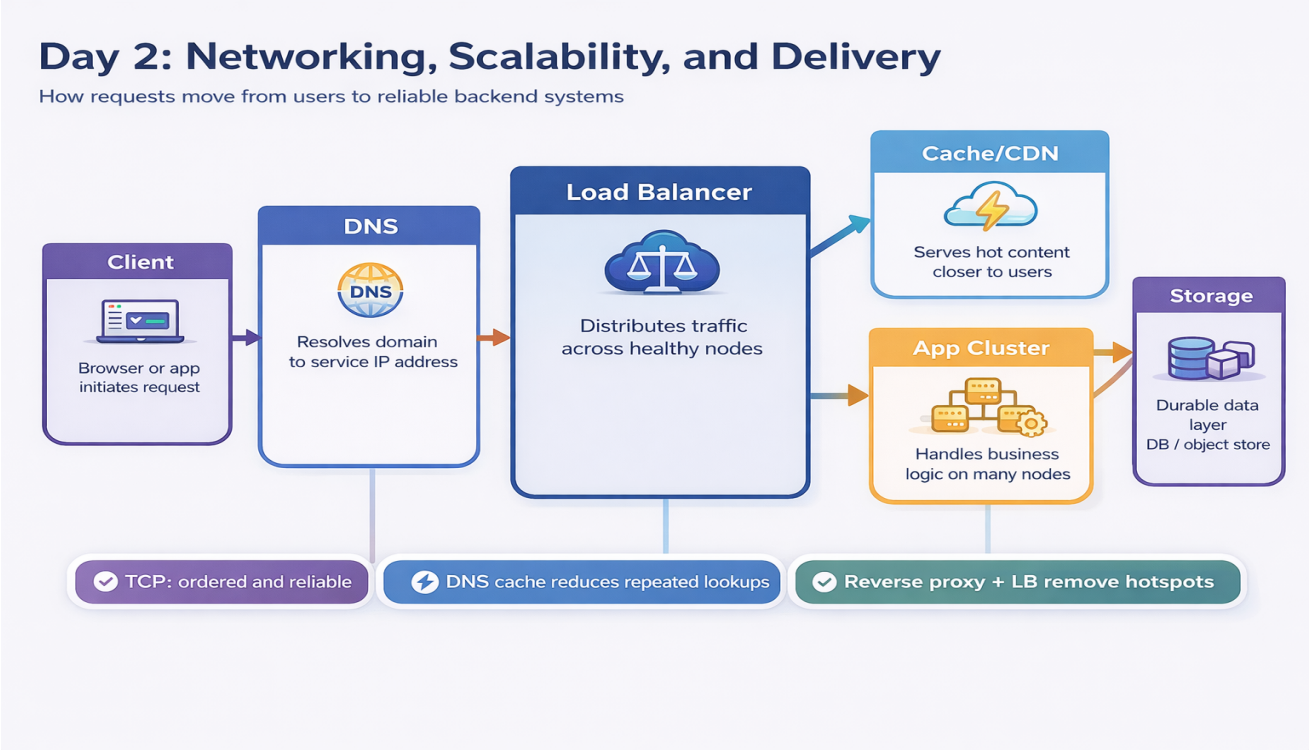
Day 2: Networking, Scalability, and Delivery

Day 2 focuses on the infrastructure concepts that sit underneath most distributed systems.

It covers:

- IP addressing and why every machine needs a routable identity
- OSI model as a troubleshooting and communication framework
- TCP vs UDP trade-offs for reliability and latency
- DNS for name resolution
- Load balancing and clustering for scale and redundancy
- Caching, CDN, and proxies for performance and control
- Availability, scalability, and storage fundamentals

In short, Day 2 is about how requests move through a system and how we keep that path fast and resilient.



Core ideas

IP, OSI, TCP/UDP

- `IP` gives devices an address on a network.
- `OSI` gives us a layered mental model for where failures happen.
- `TCP` is reliable and ordered, but comes with more overhead.
- `UDP` is lightweight and low-latency, but delivery is not guaranteed.

DNS

- Converts human-readable names like `example.com` into IP addresses.
- Uses resolvers, root servers, TLD servers, and authoritative name servers.
- Caching matters because repeated lookups are expensive at scale.

Load balancing and clustering

- `Load balancing` spreads requests across servers.
- `Clustering` coordinates multiple nodes to improve availability or throughput.
- Common load-balancing algorithms include round-robin, weighted round-robin, and least connections.

Caching, CDN, and proxy

- `Caching` reduces read latency and database pressure.
- `CDN` moves static content closer to the user.
- `Forward proxy` sits in front of clients.
- `Reverse proxy` sits in front of servers and can also handle TLS, caching, and routing.

Availability and scalability

- Availability is usually discussed in “nines”.
- Vertical scaling adds more power to one machine.
- Horizontal scaling adds more machines and is typically better for large distributed systems.

Storage

- `File storage` is path-based and simple for shared files.
- `Block storage` is low-level and good for attached volumes.
- `Object storage` is highly scalable and common for blobs and media.
- RAID, NAS, and HDFS matter when reliability and throughput are part of the discussion.

Day-2 sample: Round-Robin Load Balancer

This repository includes a small load balancer sample in both Python and Java.

Functional requirements

- Register backend servers
- Mark a server healthy or unhealthy
- Route each incoming request to the next healthy server

High-level components

- `Client`: sends requests
- `Load balancer`: picks the next healthy backend
- `Backend servers`: handle the request
- `Health state`: prevents routing to unhealthy nodes

Data flow

1. Backend servers are registered with the load balancer
2. A request arrives
3. The load balancer picks the next healthy server in round-robin order
4. If a server is unhealthy, it is skipped
5. The selected backend handles the request

Project structure

```
Day-2/
  README.md
  load-balancer-sample/
    python/
      load_balancer.py
      demo.py
    java/
      RoundRobinLoadBalancer.java
      Main.java
```

Run the sample

Python

```
cd load-balancer-sample\python
python demo.py
```

Java

```
cd load-balancer-sample\java
javac Main.java RoundRobinLoadBalancer.java
java Main
```

Interview takeaways

- DNS reduces human complexity, not system complexity
- Load balancers remove hotspots but can also become a bottleneck if not replicated
- Caches improve reads but create invalidation and consistency trade-offs
- CDN and reverse proxies are often the first big performance multipliers in internet-scale systems
- Availability and scalability decisions are usually trade-offs, not free wins

Next improvements

- Add weighted round-robin support
- Add active health checks
- Track latency and error rate per backend
- Add sticky sessions and request tracing
- Extend the sample into a reverse-proxy simulation

Source Code Appendix

Day-2/load-balancer-sample/python/load_balancer.py

```python
from __future__ import annotations

from dataclasses import dataclass, field
from threading import Lock


@dataclass
class BackendServer:
    name: str
    healthy: bool = True


@dataclass
class RoundRobinLoadBalancer:
    _servers: list[BackendServer] = field(default_factory=list, init=False)
    _index: int = field(default=0, init=False)
    _lock: Lock = field(default_factory=Lock, init=False)

    def add_server(self, name: str) -> None:
        with self._lock:
            self._servers.append(BackendServer(name=name))

    def set_health(self, name: str, healthy: bool) -> None:
        with self._lock:
            for server in self._servers:
                if server.name == name:
                    server.healthy = healthy
                    return
        raise ValueError(f"Unknown server: {name}")

    def next_server(self) -> str:
        with self._lock:
            if not self._servers:
                raise RuntimeError("No backend servers registered")

            total = len(self._servers)
            for _ in range(total):
                server = self._servers[self._index]
                self._index = (self._index + 1) % total
                if server.healthy:
                    return server.name

        raise RuntimeError("No healthy backend servers available")

    def snapshot(self) -> list[tuple[str, bool]]:
        with self._lock:
            return [(server.name, server.healthy) for server in self._servers]
```

Day-2/load-balancer-sample/python/demo.py

```python
from load_balancer import RoundRobinLoadBalancer


def main() -> None:
    balancer = RoundRobinLoadBalancer()
    for name in ("api-1", "api-2", "api-3"):
        balancer.add_server(name)

    print("Initial backends:", balancer.snapshot())
    print("Routing 6 requests:")
    for i in range(1, 7):
        print(f"  request-{i} -> {balancer.next_server()}")

    balancer.set_health("api-2", False)
    print("\nAfter marking api-2 unhealthy:", balancer.snapshot())
    print("Routing 6 more requests:")
    for i in range(7, 13):
        print(f"  request-{i} -> {balancer.next_server()}")


if __name__ == "__main__":
    main()
```

Day-2/load-balancer-sample/java/RoundRobinLoadBalancer.java

```java
import java.util.ArrayList;
import java.util.List;
```

```java
public class RoundRobinLoadBalancer {
    private static class BackendServer {
        private final String name;
        private boolean healthy;

        private BackendServer(String name) {
            this.name = name;
            this.healthy = true;
        }
    }

    private final List<BackendServer> servers;
    private int index;

    public RoundRobinLoadBalancer() {
        this.servers = new ArrayList<>();
        this.index = 0;
    }

    public synchronized void addServer(String name) {
        servers.add(new BackendServer(name));
    }

    public synchronized void setHealth(String name, boolean healthy) {
        for (BackendServer server : servers) {
            if (server.name.equals(name)) {
                server.healthy = healthy;
                return;
            }
        }
        throw new IllegalArgumentException("Unknown server: " + name);
    }

    public synchronized String nextServer() {
        if (servers.isEmpty()) {
            throw new IllegalStateException("No backend servers registered");
        }

        int total = servers.size();
        for (int i = 0; i < total; i++) {
            BackendServer server = servers.get(index);
            index = (index + 1) % total;
            if (server.healthy) {
                return server.name;
            }
        }

        throw new IllegalStateException("No healthy backend servers available");
    }

    public synchronized String snapshot() {
        StringBuilder sb = new StringBuilder("[");
        for (int i = 0; i < servers.size(); i++) {
            BackendServer server = servers.get(i);
            if (i > 0) {
                sb.append(", ");
            }
            sb.append(server.name).append(":").append(server.healthy ? "healthy" : "unhealthy");
        }
        sb.append("]");
        return sb.toString();
    }
}
```

Day-2/load-balancer-sample/java/Main.java

```java
public class Main {
    public static void main(String[] args) {
        RoundRobinLoadBalancer balancer = new RoundRobinLoadBalancer();
        balancer.addServer("api-1");
        balancer.addServer("api-2");
        balancer.addServer("api-3");

        System.out.println("Initial backends: " + balancer.snapshot());
        System.out.println("Routing 6 requests:");
        for (int i = 1; i <= 6; i++) {
            System.out.println("  request-" + i + " -> " + balancer.nextServer());
        }

        balancer.setHealth("api-2", false);
        System.out.println();
        System.out.println("After marking api-2 unhealthy: " + balancer.snapshot());
        System.out.println("Routing 6 more requests:");
        for (int i = 7; i <= 12; i++) {
```

```
            System.out.println("  request-" + i + " -> " + balancer.nextServer());
        }
    }
}
```